

M2C4 Python Assignment

During this section of Module 2, you have learned more about Python. Python is a powerful programming language that serves a lot of purposes. To practice what you have learned in this section, you will complete some Python exercises. You may use Visual Studio Code, Repl.it, or another text editor/environment of your choice. Please complete the following assignment and reach out on the Support App to have a mentor review your work. If you have any questions or need any help, please reach out so we can help you! This assignment must be completed to pass this section of the coursework.

Las preguntas teóricas:

¿Cuál es la diferencia entre una lista y una tupla en Python?

Existen varias diferencias entre la lista y la tupla tales como la sintaxis (paréntesis o corchetes) o el uso que se le da a cada estructura. Sin embargo, la diferencia más reseñable y que origina las discrepancias en el comportamiento entre la lista y la tupla sería la mutabilidad.

- Las listas son **mutables**, lo que significa que es posible modificar sus elementos después de haber sido creadas. Es posible añadir, eliminar o cambiar elementos de una lista. Las listas serían adecuadas para bases de datos donde su contenido deba ser actualizado o modificado de manera habitual.
- Por otro lado, las tuplas se consideran **inmutables**, lo que significa que una vez que una tupla ha sido creada, su contenido no puede ser alterado de ninguna manera. Sin embargo, es posible la reasignación, creando una nueva tupla y sobrescribiendo el nombre de la variable que se le da. De esta manera se crearía una nueva tupla, es decir, con otra id diferente a la creada originalmente. Las tuplas son recomendables en aquellos casos donde los datos asignados permanecerían constantes y no se deseen modificar, asegurando la seguridad de los datos asignados a las mismas.

¿Cuál es el orden de las operaciones?

Las operaciones matemáticas en Python seguirán el siguiente orden, también denominado PEMDAS, por los nombres de las operaciones en inglés:

- 1º Paréntesis: ()
- 2º Exponenciales: ** y % (módulo)
- 3º Multiplicación: *
- 4º División: /
- 5º Suma: +
- 6º Resta: -

Si en la ecuación se encuentra un paréntesis se realizará esa operación en primer lugar, en caso de no haber paréntesis se realizará la operación exponente, si no la multiplicación y así seguidamente hasta la resta.

¿Qué es un diccionario Python?

Es una estructura mutable de datos que asocia un/os valor/es a una clave, se determina entre los símbolos {'clave': 'valor' }. Puede albergar varias entradas de clave asociadas cada una a uno o varios valores. A continuación, se exponen ejemplos de diccionarios y algunas operaciones con esta estructura:

Creación de un diccionario

```
persona = {  
    'nombre': 'Carlos',  
    'edad': 40,  
    'profesión': 'Abogado'  
}
```

Acceso a valores

```
print(persona['nombre']) # Salida: Carlos
```

Modificación de valores

```
persona['edad'] = 41
```

Adición de nuevos pares clave-valor

```
persona['ciudad'] = 'Sevilla'
```

Eliminación de un par clave-valor

```
del persona['profesión']
```

Métodos útiles

```
claves = persona.keys() # dict_keys(['nombre', 'edad', 'ciudad'])
```

```
valores = persona.values() # dict_values(['Carlos', 41, 'Sevilla'])
```

```
elementos = persona.items() # dict_items([('nombre', 'Carlos'), ('edad', 41), ('ciudad', 'Sevilla')])
```

```
print(claves)
```

```
print(valores)
```

```
print(elementos)
```

¿Cuál es la diferencia entre el método ordenado y la función de ordenación?

1. Método .sort():

El método .sort() se utiliza para ordenar listas en su lugar. Esto significa que modifica la lista original y no devuelve una nueva lista ordenada.

Características del método .sort():

- Modificación en su lugar: Modifica la lista original.
- Retorno: No devuelve ningún valor (retorna None).
- Sintaxis: `lista.sort(key=None, reverse=False)`

2. Función sorted()

La función sorted() se puede utilizar para ordenar cualquier iterable (como listas, tuplas, cadenas, etc.) y devuelve una nueva lista ordenada, dejando el iterable original sin cambios.

Características de la función sorted():

- No modifica el original: No modifica el iterable original.
- Retorno: Devuelve una nueva lista ordenada.
- Sintaxis: `sorted(iterable, key=None, reverse=False)`

¿Qué es un operador de reasignación?

Es un operador que combina una operación aritmética con una asignación. Permitirán realizar una operación sobre una variable y luego reasignar el resultado a esa misma variable. Por ejemplo:

```
variable_num = 5
```

```
variable_num += 3 #Equivalente a: variable_num = variable_num + 3
```

```
print(variable_num) → #Salida: 8
```

En esta operación descrita se ha reasignado el valor de la variable_num de 5 a 8. Se podrán utilizar otros operadores de reasignación aparte del de suma descrito arriba, se exponen algunos a continuación: suma +, resta -, multiplicación *, división /, floor división //, exponente ** o módulo %.